ASSIGNMENT 1

PAYMENT CALCULATOR FOR A VILLAGE MILK CO-OPERATIVE

Task 7: Build a Module

Aim: To store rate constants and calculation functions in a module and use them from another Python script.

Module: The file `milkrate.py` contains the rate constants, `rate_per_litre()` and `payout()`. The file `task07_main.py` imports the module and calls its functions.

Import methods:

`import milkrate` imports the module and uses names such as `milkrate.payout()`.

`from milkrate import payout` imports the `payout()` function directly.

`import milkrate as mr` imports the module using the shorter alias `mr`, allowing calls such as `mr.payout()`.

Main guard: `if __name__ == "__main__":` runs the demonstration code only when `milkrate.py` is executed directly. When another file imports it, the demonstration does not run automatically.

Standard modules: The `decimal` module provides decimal arithmetic suitable for monetary calculations. The `math` module provides mathematical functions such as `floor()`.

Module search: Python looks for modules in locations listed in `sys.path`, including the current script's directory. Keeping both files in the same folder allows the main script to import `milkrate.py`.

Alternative concept: The calculation functions and constants could be duplicated in the main script instead of using a module. However, this creates repeated code and makes rate changes harder to maintain. A shared module keeps the rate rules in one place.

Complexity: Calculating one payment takes O(1) time and O(1) auxiliary space. Importing a module has a one-time loading cost; using the functions does not change the calculation's O(1) complexity.

Conclusion: Separating the rate card and calculation functions into a module improves reuse, readability and maintenance.

COURSE CODE: CSA08 STUDENT NAME: DARA KUSUMANJALI
FACULTY: POKURI SWETHA REGISTER NUMBER: 192425398L

